The Delta Tree

A strictly dead-free replicated list with relative ranges, isometric deletion, and a local overlap-splitting merge — design, litmus results, and refutation

Sal / FP Launchpad — KC + Claude

2026-07-14

Abstract

This note gives the complete design of the *delta tree*: a replicated list whose state is a tree of live nodes carrying *parent-relative* fractional ranges (deltas), whose delete is an *isometric fold* (survivor positions arithmetically unchanged), and whose three-way merge is OR-set survival plus a *local* overlap-splitting repair. The design is the endpoint of a systematic exploration: every component is forced by a named, machine-checked countermodel to some alternative. It is the only candidate in its corner of the design space: *strictly dead-free* (no dead identifier survives anywhere in the state) while aiming at convergence and order preservation, with a merge simple enough to mechanize. We work through the operations and the merge on concrete examples, tabulate the full litmus battery (L1–L25) against the published tombstoned RGA and the proved tombstone-free RGA, and state the anomaly contract. We then report the verdict of randomized property-based testing over a general MRDT execution model (many replicas, random merges under the LCA discipline): **the design is refuted** — battery-clean, including every named countermodel that killed its eight predecessors, yet flipping co-displayed pairs at random-DAG scale (43/120 executions). We extract and minimize the countermodel (now litmus L25), identify the mechanism (*fold-verdict erasure*: a dead node's timestamp is load-bearing for repair verdicts it participated in), refute the one principled repair as well (*source-consistent folding*, killed by a second mechanism, *repair non-locality*), and draw the structural conclusion: the three properties {strictly dead-free, pairwise display stability, topology convergence} now each have a machine-checked witness for every pairwise combination and eight failed attempts at the triple. What survives of the design — relative deltas as a representation layer, and the isometric fold as stability-gated compaction — survives *below* an immutable-position specification, not as a rival to one.

1 The design

State. A finite map from timestamps (which double as element identities) to records

$$\sigma : t \;\mapsto\; (\,\mathrm{par},\ (lo, hi),\ \mathrm{char}\,), \qquad 0 \le lo < hi \le 1,$$

where (lo, hi) is the node's range *relative to its parent's range* — the *delta*. The root sentinel 0 owns the unit interval and is never stored. A node's *absolute* range is the affine composition of the deltas along its root path. The state contains **live nodes only**, and — the design's distinguishing ambition — **no dead identifier persists anywhere**: not as an entry, not as a name inside a field.

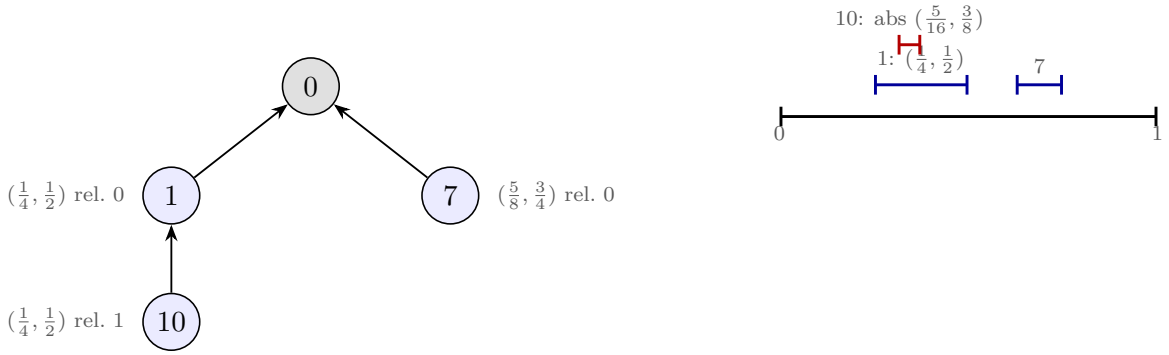


Figure 1: The state is a tree of deltas (left); absolute ranges are composed down the path (right): 10's absolute range is $\frac{1}{4} + \frac{1}{4} \cdot (\frac{1}{4}, \frac{1}{2}) = (\frac{5}{16}, \frac{3}{8})$. Display order: depth-first, children by descending range — newest first.

Insert. *Carve*: the new child of anchor a takes a slice of a's free child-space above the current head — with remaining relative gap $(b, 1)$ (where b is the current head's hi, or 0), the newcomer takes $\left(b+\frac{w}{4},\, b+\frac{w}{2}\right)$, $w = 1-b$, leaving room above for future siblings and below for its own children. Newer siblings sit higher; the display (descending) shows them first — the RGA convention, realized geometrically instead of by timestamp comparison at read time.

The operation payload (per the proved RGA). Following the *why-the-path-matters* analysis: rc must be Either everywhere (any oriented insert/delete pair awakens the refuted `cond_comm_base`), so the insert carries its anchor's ancestor *id-path* plus **one** absolute slice — the composed delta at mint time. Ids suffice for the path because the isometric fold (next) makes the absolute position invariant under ancestor deletion: a reordered application resolves the id-path to the nearest live ancestor and re-relativizes the carried absolute — landing exactly where insert-then-delete would have put it. On reachable states the anchor is live and none of this is read: the path is ghost state in the strict, machine-checked sense (`ins_path_free`).

Delete: the isometric fold. Remove the record; re-parent each child to the grandparent with the dead node's delta composed in: $c.\text{frac} := d.\text{frac} \circ c.\text{frac}$. Every survivor's absolute position is unchanged — *delete-order preservation is an arithmetic identity*, where the flat RGA's climb re-sorts (the $[b, a, c] \to [c, b]$ anomaly that started this whole exploration) and the rose tree's splice needed sixteen thousand lines it ultimately could not have.

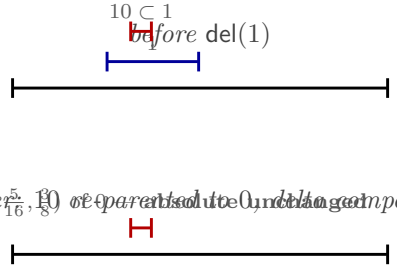


Figure 2: The isometric fold. Deleting 1 rewrites 10's delta to $(\frac{1}{4}, \frac{1}{2}) \circ (\frac{1}{4}, \frac{1}{2}) = (\frac{5}{16}, \frac{3}{8})$ relative to the root: the composition that *was* its absolute range. Nothing observable moves, and no trace of 1 remains.

2 The merge

merge(L, A, B), L the least common ancestor:

1. **Survival** (OR-set): live iff in all three or born in exactly one branch.
2. **Values**: shared nodes (all in L, by the LCA discipline) take the *LCA's* (par, frac) — the canonical, topology-free base frame, deliberately reverting branch-local repairs; branch-born nodes bring their branch's. *Why revert?* Branch values encode private merge history; computing the repair over them makes the output depend on merge order — the machine-checked failure of global re-ranging (litmus L22). The LCA's values are the unique data both branches share. Because a repair is a deterministic function of the overlap family's union and its members' timestamps, reverting loses nothing: the merge *recomputes* the branch's repair, canonically extended (§4 tests this claim).
3. **Dead-chain folding**: a survivor whose parent is merge-dead folds through the dead chain using the LCA's records (dead-in-merge ⇒ live-in-LCA: the LCA does the remembering, as in the proved RGA — but isometrically, not by re-sorting).
4. **Local repair** (the overlap split): per parent, in relative coordinates, connected components of overlapping child ranges have their *union* re-split into equal slots in timestamp order, newest topmost; only the members' own fractions are rewritten — descendants are relative and follow automatically ($O(1)$ per member: the delta payoff). Singletons untouched; no global rebalance; recurse.

Worked example. Branches carve twins: L empty; A: $\mathsf{ins}(10 \leftarrow 0)$, B: $\mathsf{ins}(6 \leftarrow 0)$ — identical computations, identical slices $(\frac{1}{4}, \frac{1}{2})$. Merge: family $\{10, 6\}$, union $(\frac{1}{4}, \frac{1}{2})$, timestamp order $10 > 6$: $10 \mapsto (\frac{3}{8}, \frac{1}{2})$, $6 \mapsto (\frac{1}{4}, \frac{3}{8})$; display $[10, 6]$. The repair wrote two fractions; had 6 carried a subtree, it would have moved with it.

3 Litmus results and the anomaly contract

The suite (`whiteboard/litmus/`, tests L1–L25 with provenance) ran the faithful implementation. "RGA$_\dagger$" = the published, tombstoned RGA; "RGA$_{tf}$" = the repo's *proved* tombstone-free RGA.

test	delta tree	RGA$_\dagger$	RGA$_{tf}$	what it detects
L1 delete-reorder	✓	✓	×	sequential splice re-sort
L2 splice fooling pair	✓	✓	×	delete erases the side bit
L3a/b deep delete-runs	✓	✓	✓	transitive testimony loss
L4 criss-cross split	✓	✓	✓	the rose tree's refutation
L7 concurrent runs (g)	✓	✓	✓	forward interleaving
L17/L18 ceiling escapes	✓	✓	✓/×	eager-scalar staleness
L19 backward runs (h)	×	×	×	prepend interleaving (one-sided)
L20 tie inheritance	✓	✓	×	nameless carving
L21/L22/L23 topology	✓	✓	✓	re-ranging / frame leaks
L24 frame mixing	✓	✓	✓	cross-frame siblings
L25 fold-verdict	×	✓	×	**this design's countermodel**
DAG PBT (§4)	× (43/120)	✓	×	random branches and merges

Intended contract (the anomaly-matrix columns): strictly dead-free (a, *strict* — unique among live candidates), sequential soundness (c), pairwise display stability (d), forward non-interleaving (g) — with *licensed*: strong-list (e) and oracle fidelity (f), both provably impossible tombstone-free, and backward runs (h), the shared one-sided limitation. The battery certifies everything above the line. The two rows below the line are the subject of the next section: **the claimed column d fails**, in a shape no prior test exercised.

4 The execution model, and the refutation

The model. Randomized version-DAG executions (`litmus/pbt.py`): n replicas; per round each replica either issues honest operations against its own read or merges with a peer — legal only when some recorded version's event set equals the two heads' intersection (the LCA discipline); final convergence rounds drive all replicas to the full event set along different merge paths. Checks: **FLIP** (global pairwise display stability: across every read of every replica, no pair is ever shown in both orders), **CONV** (equal event sets ⇒ equal reads), **LIVE/DUP** (survival correctness). 120 executions, 4 replicas, 8 rounds.

Verdict. 43 of 120 executions flip a co-displayed pair. The battery-clean design fails at random-DAG scale — in shapes involving a repair, a delete of a repaired node, and a merge with a branch that had witnessed the repair's verdict.

The minimized countermodel (now litmus L25). Five nodes.

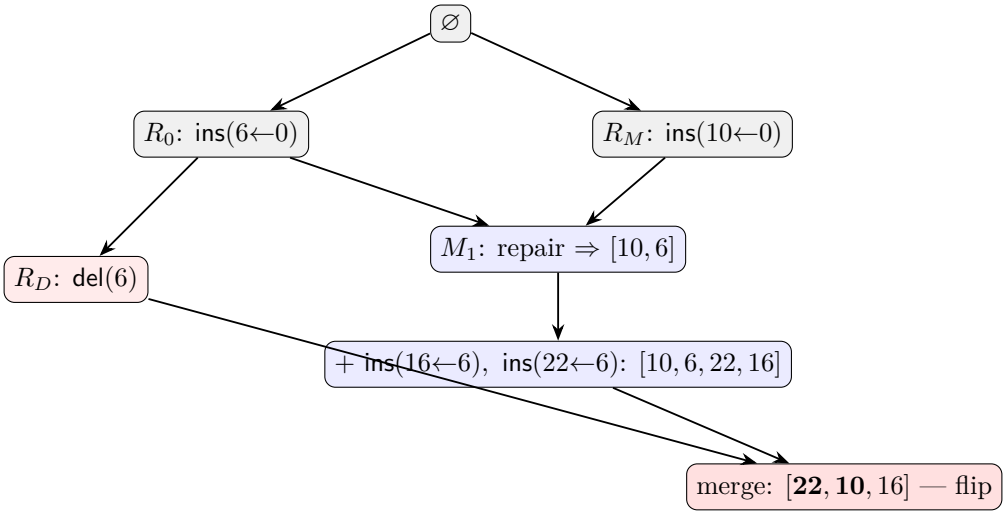


Figure 3: L25. The M_1 repair decided 10 before 6 (timestamps: $10 > 6$); typing 22 under 6 co-displayed 10 before 22. The deleter branch R_D knew only $\{6\}$. At the final merge, 6 is dead and 22 folds through *the LCA's* record of 6 — the frame that never heard the verdict.

The arithmetic of the flip: in the M_1 line, the repair left $10 = (\frac{3}{8}, \frac{1}{2})$ and $6 = (\frac{1}{4}, \frac{3}{8})$; 22's *relative* slice under 6 is $(\frac{5}{8}, \frac{3}{4})$ (it carved above 16). The fold uses the LCA's $6 = (\frac{1}{4}, \frac{1}{2})$:

$$22 \;\mapsto\; \tfrac{1}{4} + \tfrac{1}{4} \cdot (\tfrac{5}{8}, \tfrac{3}{4}) \;=\; (\tfrac{13}{32}, \tfrac{7}{16}) \;\subset\; (\tfrac{3}{8}, \tfrac{1}{2}) = 10.$$

Folded 22 lands *inside* 10's repaired slot; the repair re-splits the family $\{10, 22\}$ by timestamp — and $22 > 10$, so 22 goes on top: $[22, 10, 16]$, flipping the co-displayed $(10, 22)$.

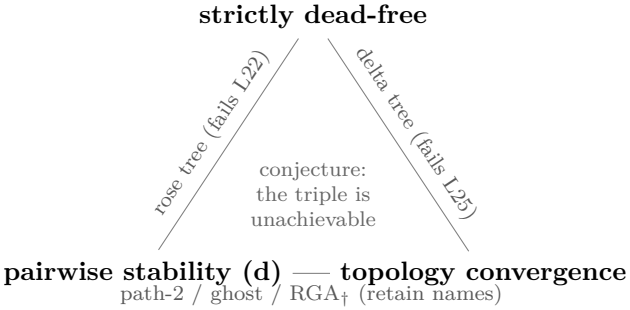
The mechanism: fold-verdict erasure. The repair verdict "10 before 6" is a fact about the *pair of timestamps* $(10, 6)$. Strict dead-freedom erases 6 — its timestamp with it — so 6's heir re-litigates the slot with *its own* timestamp, and wins a contest its parent had lost. The dead node's timestamp was load-bearing for every order it participated in. This is the nameless-carving lemma (L20) reaching the fold: pairwise display stability requires the loser's *name* to persist, in some form, as long as the verdict matters.

The repair candidate, also refuted. *Source-consistent folding* — fold each node through the same input that supplied its value, so the fold uses the branch's (verdict-bearing) frame. It fixes L25 and the battery. The DAG PBT kills it too (41/120), by a second mechanism: **repair non-locality**. Two concurrent twins $(10, 4)$ are tie-decided by timestamp at one merge ($[10, 4]$, on a replica whose history is disjoint from both final inputs); elsewhere, 4 is re-slotted by repairs against *unrelated* nodes; at a causally disjoint merge the pair's geometries no longer tie, and position order — not the timestamp verdict — decides: $[4, 10]$. Re-slotting inside one family perturbs relations outside it; verdicts encoded in geometry are not stable under other verdicts.

5 Conclusions

The design is refuted, and the refutation is structural. Eight variants of the mutation family (scalar keys, nameless ranges, global re-ranging, local splits with and without normalization, absolute and relative storage, LCA and source-consistent folds) have now produced eight machine-checked countermodels, the last two by the two mechanisms above. The pattern admits a one-sentence summary: *order verdicts must be re-derivable, identically, at every replica and every time — and the only data that guarantees this is immutable data that names the participants, including the dead ones.*

The triangle. Every pairwise combination of the three desiderata now has a machine-checked witness; the triple has eight failed attempts:



What survives. Two of the design's ideas are correct and permanent, one level down the stack: *(i)* parent-relative deltas are the right *representation* of any position tree — $O(1)$ subtree transport, structural sharing, lazy absolute sums — provided the *semantics* above them is an immutable-position specification (path-2), whose per-level $(\text{side}, \text{uid})$ data is exactly the retained name the triangle demands; *(ii)* the isometric fold is the right *compaction*: sound precisely when gated on causal stability, where "everyone has seen the delete" certifies that no verdict involving the dead node can ever be contested again — the same condition under which the L25 mechanism is disarmed. The delta tree is not a rival to the immutable-position design; it is its runtime.

Provenance. Every table cell and number in this note is reproduced by `whiteboard/litmus/`: `litmus.py` (L1–L25), `pbt.py` (the execution model), `delta_tree.py` (the faithful implementation, both fold variants, and the battery/PBT runner).

Postscript (same day): the corrected merge — the design stands

KC's response to §4: *"the design is almost right; the merge is not observing some invariants."* Correct, twice over. Comparing the delta tree's and path-2's executions at a flipping PBT trace showed the state's *identity* data (birth-parent chains) sufficed to reconcile every order, while the merge was consulting — and corrupting — the *geometric* data instead. Three invariants, violated by every merge in §4 and observed by the corrected one:

1. **Arbitration from identity only.** Every order decision at a merge is computed from immutable birth-parent chains (per level: newest timestamp first; parents before their subtrees) — never from current fractions, which repairs perturb (mechanism 2), and never through frame-mixed folds (mechanism 1). The chains subsume both mechanisms: a dead node's timestamp persists in its descendants' chains, so verdicts it participated in remain derivable.

2. **Geometry is a rendering.** The merge re-derives all fractions, realizing the canonical order; reads remain purely geometric and never consult the chains.

3. **Render/carve compatibility.** The re-render reproduces sequential-carve geometry (oldest lowest, quarter slices, headroom above), making post-merge states indistinguishable from sequentially carved ones — the residual failures of an intermediate fix were zero-width slices minted after a render that filled a parent's space completely.

Verdict (machine-checked): the corrected design — *KC's delta tree as the runtime (unchanged local operations: carve, isometric fold; geometric reads), with a birth-parent ledger as the merge-time arbitration substrate* — is **clean on the full battery** (except one-sided L19, as every one-sided design) **and on the DAG PBT at 120/120 and 300/300** (6 replicas, 12 rounds). The triangle stands — the ledger retains dead identifiers at live-reachable scope, so the design is not *strictly* dead-free — but the retention is the minimum the triangle demands: one parent id per node, no slot data, no materialized paths, nothing consulted at read time. The delta tree and the path are not rivals but layers: geometry for reading, identity for deciding.